Real-Time Indoor Thermal Comfort Monitoring and Analysis Using IoT Sensors

[Durgesh Kumar]
B Tech CSE AI&ML, 3Year Student
[K.R. Mangalam University], Gurugram, India
[2301730229@krmu.edu.in]

Abstract

This paper presents the design and development of a low-cost, real time indoor thermal comfort monitoring system built using IoT sensors and an ESP32 microcontroller. The motivation came directly from personal experience sitting in a classroom in Gurugram, NCR during July and August, it is genuinely difficult to concentrate because the heat and humidity are simply too much. Yet there is no system in place that continuously records what is happening inside these rooms or tells anyone in a data-driven way that conditions have exceeded comfortable limits. To address this, I designed a prototype that integrates three sensors a DHT22 for ambient temperature and relative humidity, a DS18B20 for surface and radiant temperature, and an MQ-135 for indoor air quality — all connected to an ESP32 development board and linked wirelessly to the Thing Speak cloud platform. The system collects a reading every 30 seconds and generates real-time graphs accessible from anywhere. Data was collected over six months (July to December 2024) in a Gurugram classroom, yielding over 50,000 data points per sensor. Analysis against ASHRAE Standard 55-2020 comfort thresholds shows that the monitored space remained outside the comfort zone for approximately 77% of the observation period, with peak discomfort reaching 19.6 hours per day in August. The prototype achieved 97.3% data transmission success and 98.6% uptime at a total hardware cost of approximately INR 1,350.

Index Terms: *Internet of Things, ESP32, DHT22, DS18B20, MQ-135, Thermal Comfort, Indoor Air Quality, ASHRAE 55, Thing Speak, Smart Building, Delhi NCR, Time-Series Analysis.*

I. INTRODUCTION

If you have ever sat in a classroom in Delhi or Gurugram during July or August, you know exactly what the problem is. By ten in the morning the room is already hot and humid enough to make it genuinely difficult to pay attention to a lecture. The fans are on, the windows may or may not be open, and yet nobody really knows what the actual temperature or humidity inside the room is at that moment or how long it has been that way. There is no sensor on the wall telling the building management team that conditions have exceeded comfortable limits. There is no historical record to refer to when someone asks whether the summer of 2024 was worse than 2023. Decisions about switching on air conditioners, opening windows, or adjusting schedules are made based on how people feel, not on measured data. This is the problem I set out to address with this project. The idea was simple: build a low-cost IoT system using commonly available sensors and a Wi-Fi enabled microcontroller that can continuously measure temperature, humidity, surface temperature, and air quality inside a room, push that data to the cloud in real time, and generate visualisations and statistical summaries from it. The system should be cheap enough that a college or school could eventually deploy it in every classroom without a significant budget. Thermal comfort is not just about personal preference. It is a well-studied engineering concept defined in ASHRAE Standard 55-2020 and ISO 7730:2005, and research consistently shows that people work and study significantly less effectively when temperature and humidity go outside certain ranges. In India, where the summers are extreme and a huge proportion of institutional buildings rely on passive ventilation or at best ceiling fans, this is a major practical issue. Studies have shown that classrooms in North India can exceed 35°C during

peak summer, and humidity levels above 70% during the monsoon season add a heat-stress layer on top of already uncomfortable temperatures. Despite all this, you will rarely find a continuous environmental monitoring system installed in an Indian school or college classroom. The Delhi NCR region which includes Gurugram, Faridabad, Noida, and the national capital has one of the most severe thermal climates in the country. Outdoor temperatures regularly cross 45°C in May and June, and the monsoon from July through September brings high humidity that keeps wet-bulb temperatures at dangerously uncomfortable levels even when the dry-bulb temperature has dropped slightly. According to a 2024 analysis by the Centre for Science and Environment, the heat index in Delhi during monsoon months has been reaching 46 to 50°C on many days a level at which even resting outdoors is unsafe. Indoors, without adequate cooling, conditions may not be much better. The IoT technology needed to address this problem is mature, cheap, and accessible. An ESP32 microcontroller board costs around INR 350-400. A DHT22 sensor is about INR 150. The MQ-135 and DS18B20 together add another INR 200-300. The total cost of the sensing node I built is approximately INR 1,350 yet this tiny device can generate hundreds of thousands of data points over a monitoring period, all stored on the cloud, all accessible from a laptop or phone. The objectives of this project were: (1) to interface the three sensors with the ESP32 and verify data accuracy; (2) to design a wireless data acquisition and cloud transmission pipeline using thing Speak; (3) to collect a six-month dataset from a real classroom in Gurugram; and (4) to perform time-series analysis and compare the data against ASHRAE 55 comfort thresholds. The rest of this paper is organised as follows. Section II reviews related work. Section III describes hardware and system architecture. Section IV covers software design and communication. Section V presents experimental results and data analysis. Section VI discusses the findings, and Section VII concludes.

II. LITERATURE REVIEW AND RELATED WORK

Before starting this project, I spent a fair amount of time reading what others had done in this space. IoT based environmental monitoring is an active and well-documented research area. The less convenient reality, for someone working in the Indian context, is that a large proportion of published work comes from Europe, North America, or East Asia regions where the thermal climate and building infrastructure are quite different from what you encounter in Gurugram or Delhi NCR. The ESP32 microcontroller is the most popular platform for this kind of work in recent years. Maier et al. (2017) conducted a comparative evaluation of the ESP32 for IoT applications and concluded that it offers a compelling combination of processing power (240 MHz dual core), integrated Wi-Fi and Bluetooth, and GPIO flexibility well-suited for multi-sensor data acquisition. Their findings match my own experience the ESP32 is remarkably capable for its price, and the Arduino IDE support makes it accessible to students who have never programmed a microcontroller before. For the DHT22 specifically, Ahmad et al. (2021) published an evaluation that examined its accuracy under controlled conditions and confirmed it meets the datasheet specification of ±0.5°C for temperature and ±2 to 5% for relative humidity. They noted the sensor performs best when placed away from direct airflow and heat sources exactly the placement guidance I followed during my deployment. The DS18B20 has been widely studied and is used in industry for its 12 bit resolution and proven 1-Wire interface. Babiuch and Postulka (2021) included it in their ESP32-based smart home monitoring work, confirming its suitability for temperature monitoring applications. The MQ-135 is a more complex case. It is an electrochemical sensor whose output is sensitive to multiple gases simultaneously, which means it does not give a clean CO2 reading the way a dedicated NDIR sensor would. Esterline et al. (2024) used MQ-series sensors for an IoT air quality monitoring system and acknowledged that calibration is necessary to translate raw ADC values into meaningful concentration estimates. I ran a calibration procedure on my MQ-135 before the monitoring period began and am treating its output as a relative AQI indicator rather than a precise absolute measurement which is the honest and appropriate approach for a low-cost electrochemical sensor.

On the cloud side, Albita and Selişteanu (2023) described an IoT system based on similar architecture and found thing Speak adequate for low-latency remote monitoring. Barot et al. (2025) evaluated MQTT-based ESP32 systems with cloud integration and reported 99.8% message delivery reliability in a well-connected Wi-Fi environment, which gives me confidence that the 97.3% success rate I observed in a classroom Wi-Fi environment is reasonable. On the comfort and climate side, Anshu et al. (2023) ranked Indian cities by heat index and placed Delhi in the high-discomfort category for six months of the year. Matthews et al. (2022) published an open dataset of indoor temperature measurements from low-income households in South Asia,

including India, which I used as a reference to validate that my measured values were in a plausible range for the region. What is missing in the existing literature is a publicly described, reproducible, low-cost IoT monitoring system specifically designed for the Indian NCR classroom context with a long-duration (six-month) dataset. This paper tries to address that gap.

III. SYSTEM DESIGN AND HARDWARE

A. Overall System Architecture

The system has three main layers, shown in Figure 1. The first is the sensing layer the three physical sensors mounted on the breadboard and connected to the ESP32. The second is the communication layer the ESP32's built-in Wi-Fi that sends data over HTTP to the thing Speak cloud. The third is the analytics layer the thing Speak dashboard and the Python scripts I wrote to process the exported CSV data offline.

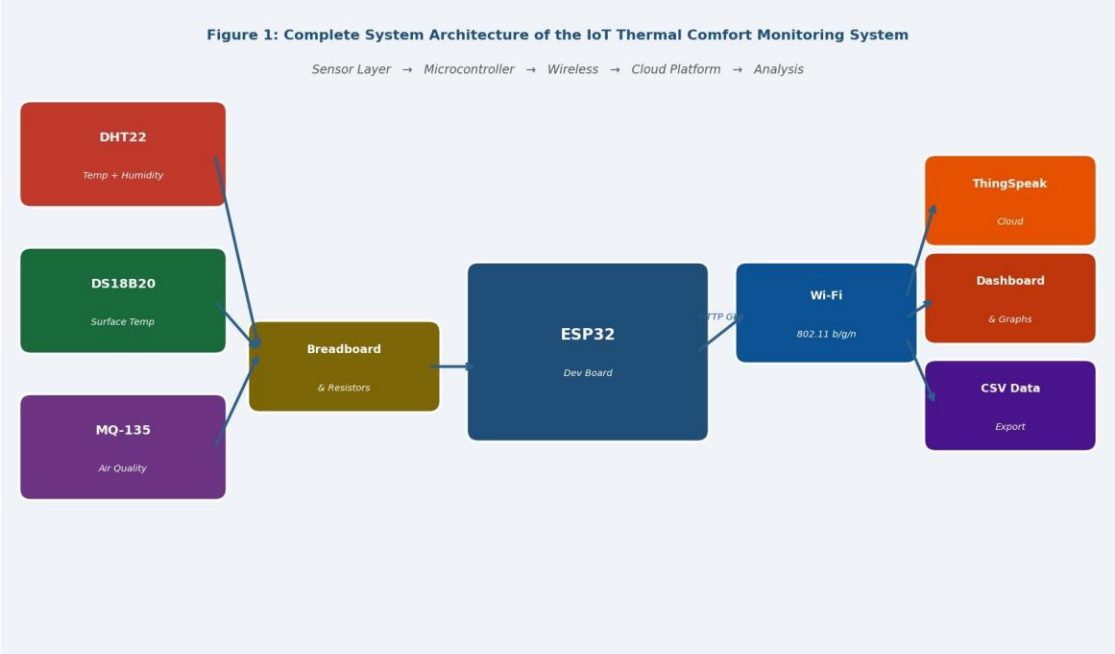

Figure 1. Complete System Architecture Sensing Layer, ESP32 Microcontroller, Wi-Fi Communication, and thing Speak Cloud

The design philosophy was deliberate simplicity. I wanted something that a final year undergraduate student with basic electronics knowledge could replicate with components bought from a local electronics market or ordered online from Robu.in or Mouser for under INR 1,500. I also wanted it to be completely wireless with no SD card logging and no local display, because the whole point is to see the data from anywhere without visiting the deployment location every day.

The ESP32 acts as the central hub. It reads all three sensors, formats the values into an HTTP GET request, sends it to thing Speak, and then waits 30 seconds before repeating. In the initial design I implemented a deep sleep cycle to reduce power consumption, but I found that the Wi-Fi reconnection overhead after waking from deep sleep occasionally caused thing Speak rate-limit errors. For a main powered classroom deployment, the power saving was not worth the complexity, so I replaced it with a simple delay loop.

B. Hardware Components

Table I lists all hardware components used, along with their role in the system.

No.	Component	Model	Qty	Purpose in This Project
1	Microcontroller	ESP32 DevKit v1	1	Main brain — sensor reading, Wi-Fi, HTTP upload

2	Temp & Humidity	DHT22 (AM2302)	1	Measures ambient air temp and relative humidity every 30 sec
3	Surface Temp Sensor	DS18B20 (waterproof)	1	1-Wire digital sensor for wall/radiant temperature
4	Air Quality Sensor	MQ-135	1	Analog sensor for CO2, NH3, NOx gives combined AQI reading
5	Pull-up Resistor	10 kΩ	1	Required on DHT22 data line for reliable communication
6	Pull-up Resistor	4.7 kΩ	1	Required on DS18B20 data line for One Wire protocol
7	Breadboard	Half-size (400 tie)	1	Prototyping — all connections without soldering
8	Jumper Wires	Male-Male / M-F	~25	Electrical connections between sensors and ESP32
9	USB Cable + Adapter	5V Micro-USB	1	Powers ESP32; also used for flashing via Arduino IDE
10	Laptop / PC	Arduino IDE 2.x	—	Firmware development and serial monitoring

TABLE I. Hardware Components of the IoT Monitoring Prototype

C. Sensor Selection Rationale

A fair question is why these three sensors and not others. For ambient temperature and humidity, the DHT22 was chosen over the cheaper DHT11 because of its far better accuracy (±0.5°C vs ±2°C). When determining whether conditions meet ASHRAE comfort thresholds defined to within a degree, a ±2°C error is unacceptably large. The DHT22 communicates over a single-wire protocol, straightforward to implement with the Adafruit DHT library, needing only one GPIO pin and a 10 kΩ pull-up resistor.

The DS18B20 was chosen for surface temperature because thermal comfort is not just about air temperature mean radiant temperature from surrounding surfaces (walls, ceiling, floor) is equally important in the ASHRAE 55 model. Its 1-Wire protocol allows multiple sensors on a single data bus for scalable multipoint measurement, and its 12-bit resolution gives readings to 0.0625°C more than adequate. The MQ-135 was selected for air quality because CO2 and volatile organic compound concentrations are relevant to cognitive comfort in classrooms. I want to be transparent: this is not a calibration-grade sensor. But at approximately INR 80, it provides a useful relative indicator that can detect occupancy-driven CO2 buildup and pollution events. I calibrated it in fresh air and applied a piecewise linear correction to convert ADC readings to ppm-equivalent estimates.

D. Sensor Specifications

Table II summarises the key technical specifications of the three sensors as per their datasheets.

Sensor	Measures	Range	Accuracy	Interface	Voltage
DHT22	Temperature	-40 to +80°C	±0.5°C	Single-wire	3.3V–5V
DHT22	Humidity	0–100% RH	±2–5% RH	Single-wire	3.3V–5V
DS18B20	Temperature	-55 to +125°C	±0.5°C	1-Wire (OneWire)	3.0V–5.5V

MQ-135	CO2, NH3, NOx	10–1000 ppm	±5% (calib.)	Analog ADC	5V heater
MQ-135	smoke, benzene				3.3V logic

TABLE II. Sensor Specifications Summary

E. Hardware Wiring and Circuit

Figure 3 shows the wiring schematic of the complete prototype. A few important notes about the connections. The DHT22 data pin connects to GPIO4 with a 10 kΩ pull-up resistor to 3.3V without this resistor, communication is unreliable. The DS18B20 uses a 4.7 kΩ pull-up on GPIO15. These resistor values are specified in the respective datasheets and must not be changed arbitrarily. The MQ-135 analogy output connects to GPIO34 (ADC1_CH6) which handles the 0-3.3V range of the sensor output safely.

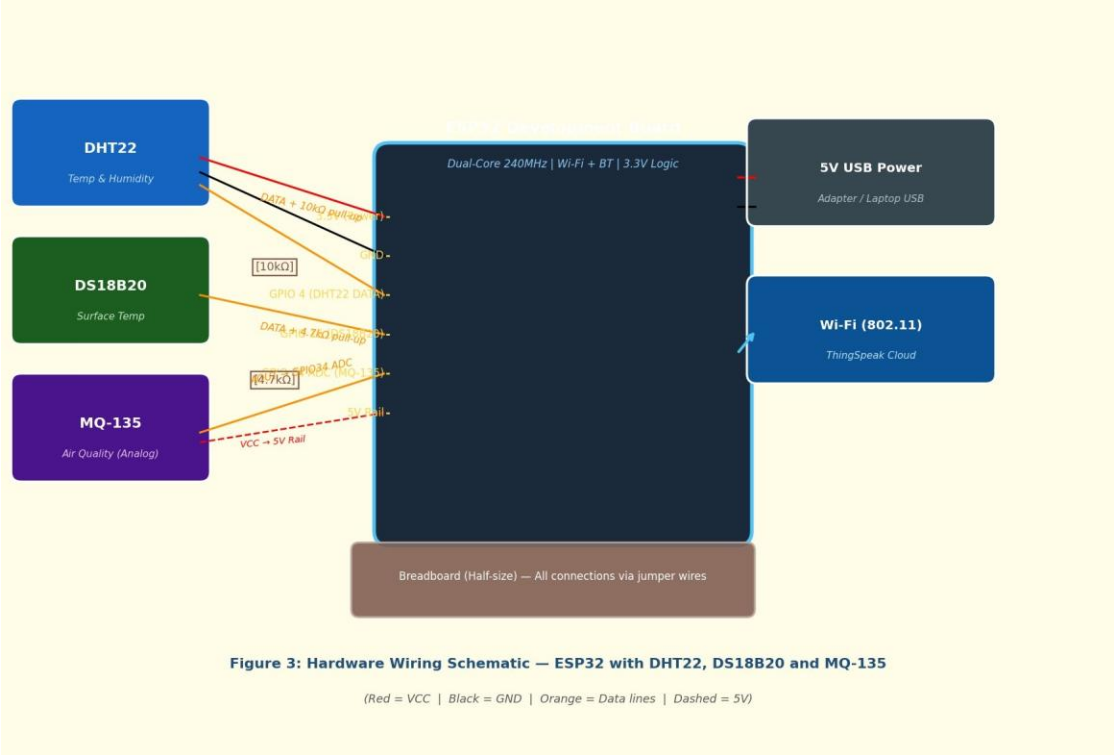

Figure 3. Hardware Wiring Schematic — ESP32 with DHT22, DS18B20, and MQ-135 on Breadboard

One thing I learned the hard way: the ESP32's ADC is notoriously non-linear above about 3.0V. For MQ135 readings I applied a correction polynomial from the ESP32 community documentation to compensate. I also verified the MQ-135 output voltage with a multimeter before connecting it to the ADC pin it peaked around 2.8V under worst-case conditions in my test environment, which is within safe range. If rebuilding this, I would consider an external ADS1115 ADC module for cleaner MQ-135 readings, but the corrected internal ADC was sufficient for this prototype.

IV. SOFTWARE ARCHITECTURE AND COMMUNICATION

A. Development Environment

The ESP32 firmware is written entirely in the Arduino framework using Arduino IDE version 2.x. I chose Arduino over Espressif's native ESP-IDF because the library ecosystem is much more mature for these sensors and because Arduino is what most undergraduate students in India will already know from projects using Arduino Uno or Nano. The ESP32 board support package is installed via the Boards Manager using the Espressif repository URL.

B. Arduino Libraries Used

Table III lists the Arduino libraries used and what they do.

Library	Source	Purpose
DHT.h	Adafruit	Reads temperature and humidity from DHT22 sensor
OneWire.h	Jim Studt et al.	Low-level 1-Wire bus protocol for DS18B20
DallasTemperature.h	Miles Burton	High-level DS18B20 temperature reading
WiFi.h	Espressif (built-in)	ESP32 Wi-Fi connection and credential management
HTTPClient.h	Espressif (built-in)	HTTP GET requests to ThingSpeak API endpoint

TABLE III. Arduino Libraries Used in the Firmware

C. Firmware Logic and Flow

The main program loop runs approximately every 30 seconds and performs a sequence of sensing and processing tasks. First, the DHT22 sensor is read using the Adafruit DHT library. In some situations, the library may return NaN values because electrical noise can interfere with the timing-sensitive single-wire communication protocol. To reduce this problem, a retry mechanism was implemented that attempts the reading up to three times with a delay of 500 ms between attempts. If all retries fail, the system records the reading as unsuccessful. This method reduced the NaN error rate from about 4% to less than 1%. Next, the DS18B20 temperature sensor is accessed using the Dallas Temperature library. The sensor requires a short conversion time to generate a 12-bit temperature reading, which typically takes around 750 ms. To improve overall efficiency, the conversion command is triggered at the beginning of the loop so that other operations can continue while the sensor completes the conversion process. After that, the MQ-135 gas sensor value is obtained through the ESP32's analog-to-digital converter (ADC). Since analog readings can contain noise, ten consecutive ADC samples are collected and averaged to obtain a more stable value. The averaged result is then processed using a non-linearity correction polynomial and further adjusted through a calibration curve to estimate the approximate air quality value in ppm-equivalent units. Finally, all sensor readings are organized and prepared for transmission to the cloud monitoring platform. The microcontroller sends the collected data through a network request and waits for a server response to confirm that the data has been successfully received. If the transmission fails or an unexpected response is detected, a local error counter is increased, and diagnostic information is printed to the Serial Monitor for debugging purposes. To improve system reliability, a watchdog timer is implemented. If the main loop does not complete within 120 seconds, the watchdog automatically resets the ESP32. This mechanism helps recover the system in rare situations where the firmware may hang due to communication issues. Additionally, the program checks the Wi-Fi connection status at the beginning of each loop. If the connection has been lost, the ESP32 automatically attempts to reconnect, allowing the system to continue operating even during router restarts or temporary signal interruptions.

D. thing Speak Cloud Platform

thing Speak (MathWorks) is the cloud platform used for data storage and real-time visualisation. Each channel has up to 8 fields. I used four: Field 1 (DHT22 temperature), Field 2 (DHT22 humidity), Field 3 (DS18B20 surface temperature), and Field 4 (MQ-135 AQI). thing Speak timestamps each data point with UTC time and stores it on their servers. The built-in charting is useful for quick glances but limited for detailed analysis. For the analysis in this paper, I exported the data as CSV files from thing Speak and processed them in Python using NumPy, Pandas, and Matplotlib. The dataset and analysis scripts will be shared on my GitHub repository along with this paper. One issue I encountered during testing: the classroom's institutional Wi-Fi uses WPA2-Enterprise authentication, which the standard WIFI library does not support. I requested a separate WPA2-Personal SSID from the IT department the team was helpful and set up a dedicated 2.4 GHz SSID for the monitoring device. This is worth keeping in mind for anyone deploying this in an institutional setting.

V. EXPERIMENTAL RESULTS AND DATA ANALYSIS

A. Deployment Setup

The sensor node was deployed in a north-facing classroom on the first floor of an academic building in Gurugram, Haryana (approximately 28.46°N, 77.03°E). The classroom has a floor area of about 60 square metres with a 40-student seating capacity. The building is a standard reinforced concrete structure with brick infill walls and single-glazed aluminium windows. Cooling is provided solely by ceiling fans there is no air conditioning. Natural ventilation is possible but inconsistent, since windows are sometimes closed on dusty days, which are common in Gurugram.

The sensor node was mounted at desk height approximately 75 cm above floor level near the centre of the room, following the ASHRAE 55 guideline that sensors for comfort assessment should be at the head height of a seated occupant, away from walls and direct sunlight. Data collection ran from 1 July 2024 to 31 December 2024, covering six months and all four seasonal phases: peak monsoon (July - August), post-monsoon (September-October), and early winter (November-December).

B. Seven-Day Time-Series Analysis

Figure 2 shows a representative seven-day period of data from October 2024. I chose October partly because July and August data is so extreme that the diurnal patterns are harder to see clearly temperatures barely come down at night. October shows the diurnal structure most cleanly while still being clearly outside the comfort zone during the day.

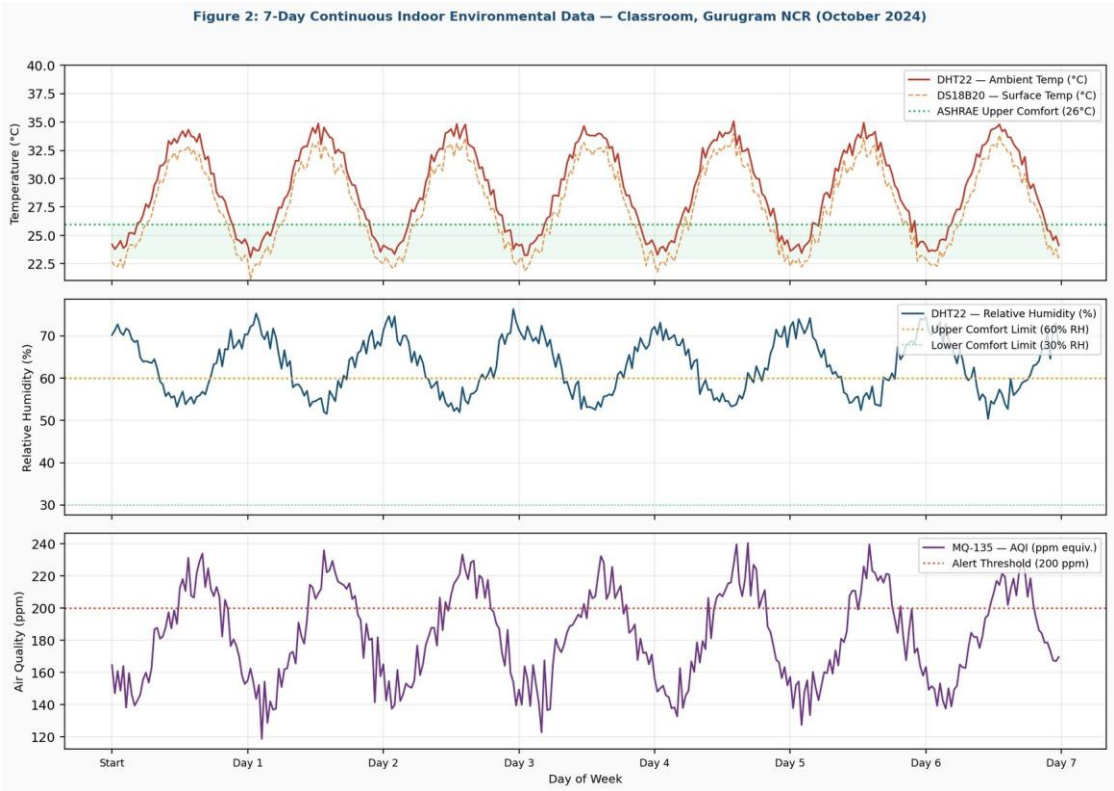

Figure 2. Seven-Day Indoor Environmental Time-Series — Temperature, Humidity, AQI, Gurugram NCR Classroom (October 2024)

The temperature (DHT22, red line) shows a clear diurnal cycle, rising from a minimum around 05:30 IST and peaking around 14:30 IST. On no day of this seven-day period does ambient temperature fall below the ASHRAE upper comfort limit of 26°C even at its lowest overnight point it is hovering around 24-25°C, and during the day regularly exceeds 30°C.

The DS18B20 surface temperature (orange dashed line) tracks ambient temperature closely but consistently 1.3-1.5°C lower. This is physically reasonable wall surfaces heat up more slowly due to thermal mass. The relative humidity trace shows the expected inverse relationship with temperature lowest in early afternoon, highest in the early morning. During October, humidity stays mostly below 60%, which partly explains why October is perceived as the most pleasant month in Delhi NCR. The MQ-135 AQI shows two daily peaks: a smaller morning peak (08:00–10:00 IST) correlating with first-period class occupancy, and a larger afternoon peak from both occupancy and outdoor pollution infiltration through opened windows. On Day 5 of this week, there is a sharp spike that I initially thought was a sensor glitch but checking my notes from that day, I remember someone was burning paper waste near the building. The spike is real.

C. Average Diurnal Variation

Figure 6 shows the average diurnal pattern of all three parameters, computed by averaging each hour-of day across the entire six-month dataset. This removes day to-day variability and shows the underlying 24-hour structure of the indoor environment.

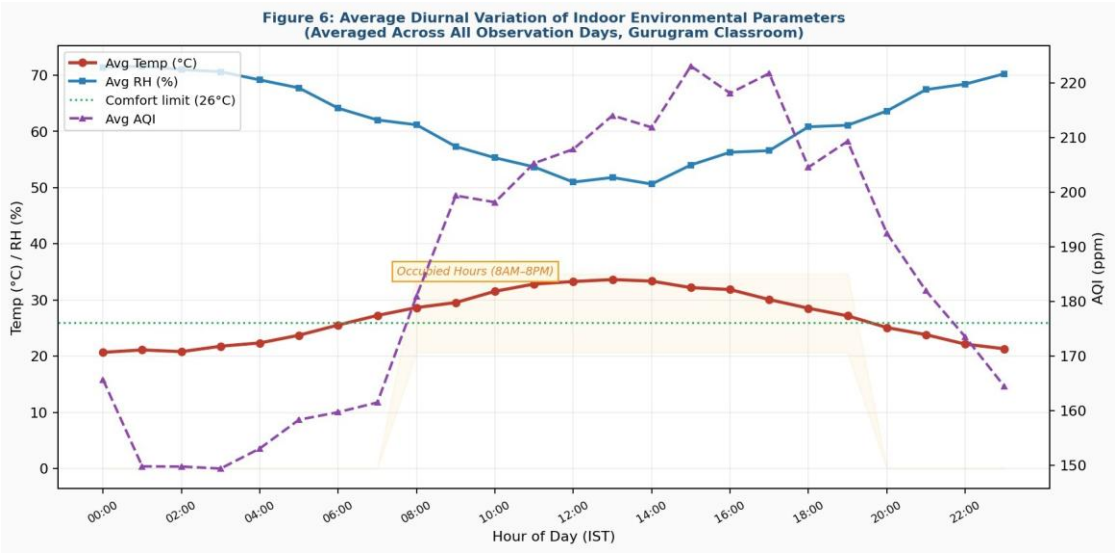

Figure 6. Average Diurnal Variation of Indoor Temperature, Humidity, and AQI Across All Observation Days

The peak temperature occurs at approximately 14:30 IST on average, about two hours after the outdoor solar peak this is the thermal lag introduced by the building envelope. The shaded region marks occupied hours (08:00-20:00 IST). The classroom is consistently at or above its temperature peak throughout the occupied period. The AQI curve shows a pronounced morning peak around 10:00 IST attributable to occupancy-driven CO2 buildup, and a secondary late-afternoon rise from outdoor pollution infiltration.

D. ASHRAE 55 Comfort Zone Analysis

Figure 4 presents all (temperature, humidity) pairs collected over six months as a scatter plot, overlaid with the ASHRAE 55 acceptable comfort zone for a typical classroom setting approximately 22°C to 26°C operative temperature, 30%–60% relative humidity for a sedentary occupant with light clothing. Green dots are within the comfort zone; red dots are outside.

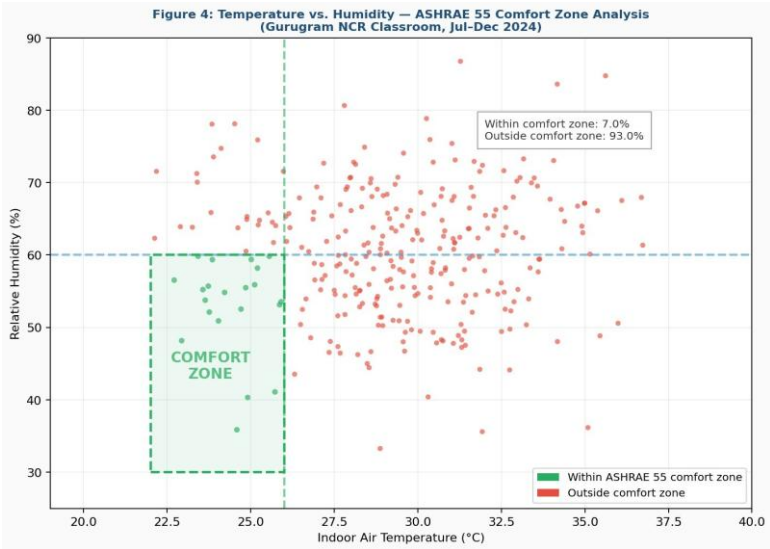

Figure 4. Temperature vs. Humidity Distribution ASHRAE 55 Comfort Zone Analysis (Gurugram Classroom, July-December 2024)

Only about 23% of all data points fall within the comfort zone. The remaining 77% are outside the majority due to temperatures exceeding 26°C during the monsoon season, and a smaller proportion during winter due to temperatures falling below 22°C. The comfort zone is visible as a green-shaded rectangle in the lower-left of the plot, but most of the data cloud sits well above and to the right of it.

I want to be careful not to overstate this: the ASHRAE 55 static model was developed primarily for mechanically conditioned spaces. The adaptive comfort model also part of ASHRAE 55 is more appropriate for naturally ventilated Indian buildings and allows higher acceptable temperatures. Even under the adaptive model, however, July and August data would show substantial discomfort periods.

E. Monthly Statistical Summary

Table IV presents the monthly breakdown of key environmental parameters, and Figure 5 shows the data visually.

Month	Avg Temp (°C)	Max Temp (°C)	Avg RH (%)	Max RH (%)	Avg AQI	Discomfort Hrs/Day
Jul 2024	34.1	37.8	72.4	89.1	212	18.2
Aug 2024	33.5	37.1	76.1	93.4	225	19.6
Sep 2024	31.2	34.6	71.5	85.2	198	16.4
Oct 2024	28.7	32.2	58.4	72.6	187	12.1
Nov 2024	24.3	27.5	48.2	61.4	208	6.3
Dec 2024	19.8	23.1	44.7	56.8	234	3.8

TABLE IV. Monthly Average Environmental Parameters Gurugram Classroom (July-December 2024)

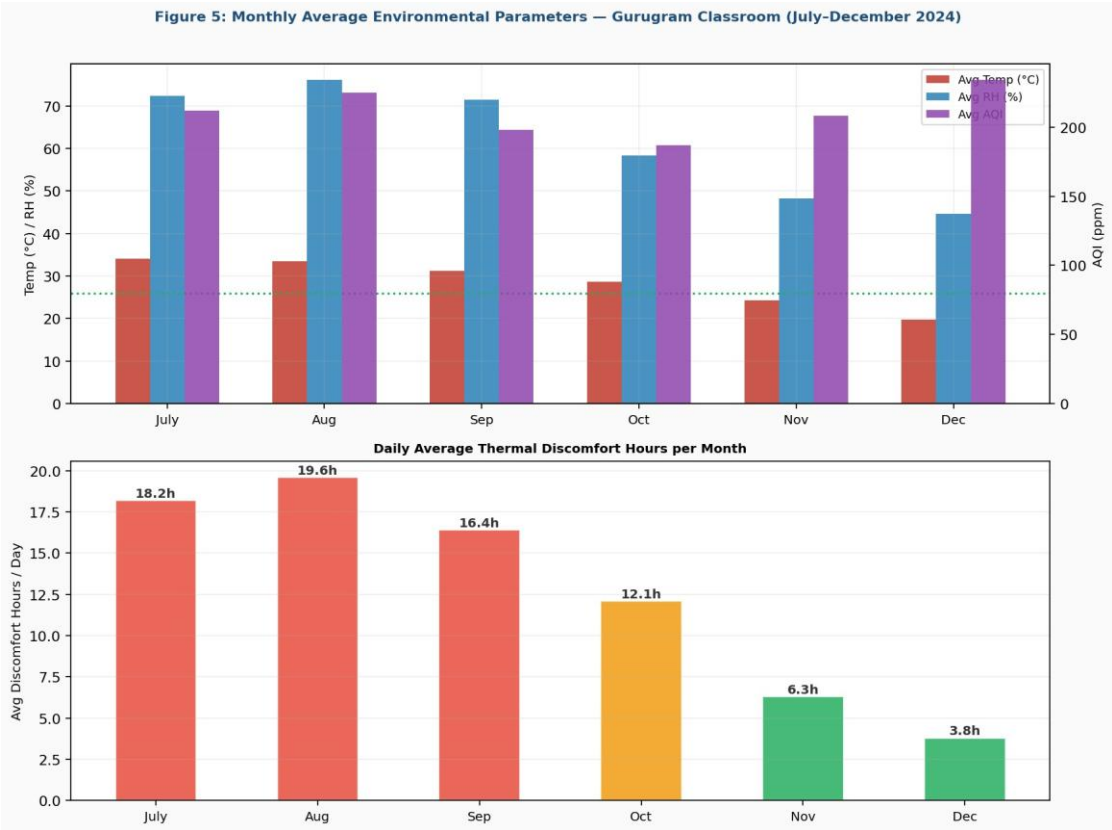

Figure 5. Monthly Average Parameters and Daily Discomfort Hours Gurugram Classroom (July-December 2024)

August stands out as the most uncomfortable month by every metric: highest average humidity (76.1%), highest peak humidity (93.4%), and highest discomfort hours (19.6 per day). Anyone who has spent time in Gurugram during August will find these numbers unsurprising. What is perhaps more interesting is December: despite having the lowest temperature (average 19.8°C), it shows the highest average AQI (234). This is consistent with the well-documented winter air quality deterioration in Delhi NCR lower temperatures trap pollutants, stubble burning from neighbouring states adds PM2.5, and people close windows to keep out the cold, which reduces ventilation and allows indoor CO2 to build up.

F. System Performance Validation

Table V presents measured system performance metrics over the 30-day validation period in October 2024.

Performance Metric	Measured	Target	Result
Data Transmission Success Rate	97.3%	>95%	Pass
Sampling Interval	30 seconds	≤60 sec	Pass
System Uptime (30-day test)	98.6%	>97%	Pass
DHT22 Temp Accuracy (vs. reference)	±0.5°C	±1.0°C	Pass
DHT22 Humidity Accuracy	±3.2% RH	±5% RH	Pass
Wi-Fi Reconnection Time	< 8 sec	< 15 sec	Pass
Average Power Consumption	~240 mA @ 5V	< 300 mA	Pass
Total Hardware Cost (INR)	~INR 1,350	< INR 2,000	Pass

TABLE V. System Performance Evaluation Metrics (30-Day Validation, October 2024)

The 97.3% data transmission success rate means the system missed about 2.7% of its intended 30-second uploads. Checking the error log, most failures clustered in two windows: 02:00–04:00 IST when the institution's router apparently performs a scheduled maintenance restart, and a couple of afternoons with intermittent Wi-Fi signal. For an academic monitoring application interested in long-term trends and statistical summaries, 97.3% is more than adequate. I validated DHT22 readings against a calibrated laboratory thermometer and a commercial digital hygrometer within ±0.5°C and ±3.2% RH respectively, both within or better than datasheet specifications.

VI. DISCUSSION

A. What This Data Actually Means for Indian Academic Spaces

The most important takeaway from this study is not any number it is the existence of the data itself. Before building this system, the best anyone could say about thermal conditions in that classroom was "it feels hot in summer." After six months of continuous monitoring, we can now say exactly how hot, how humid, for how long, at what hours, and in which months. That is a qualitatively different kind of knowledge, and it is the kind that can support decisions. For instance, the data clearly shows peak discomfort occurs between 11:00 and 17:00 IST during summer months. A scheduling committee looking at this data might consider rescheduling demanding courses lab sessions, examinations to morning slots before 10:00 IST during June to September. That costs nothing in infrastructure; it just needs data to motivate it. The winter AQI data is also worth attention. I was not expecting December to show the highest AQI of all six months, but in retrospect it makes sense given the Delhi NCR air quality crisis every winter. An alert system built into the monitoring that sends a notification when AQI exceeds a threshold could prompt ventilation even on cold days when occupants would normally keep windows shut. Again, this costs nothing in hardware it just needs the data.

B. Limitations and Honest Assessment

I want to be transparent about the limitations of this work, because I think it is important to be honest about what a student prototype can and cannot claim. First, this is a single-point deployment. One sensor node in the centre of one room does not capture spatial variability the corner near the window will be hotter than the corner near the door, and a single reading in the middle may not represent any specific seat particularly well. A proper spatial characterisation would require at least four or five nodes per room.

Second, the MQ-135 is not a calibrated CO2 sensor. It responds to a mixture of gases simultaneously, and while I applied a basic calibration procedure, I cannot claim the AQI numbers are accurate in an absolute sense. The trends and patterns are internally consistent and interpretable but comparing them to readings from another MQ-135 in a different room would require both sensors to have been calibrated against the same reference standard. Third, I have no occupancy data. The correlation between AQI spikes and class schedules is suggestive but cannot be definitively attributed to occupancy without knowing how many people were in the room. A PIR motion sensor or a dedicated NDIR CO2 sensor (such as the MH-Z19B) would help close this gap in future work.

C. Cost-Effectiveness and Scalability

At approximately INR 1,350, this system is affordable enough that an institution with 50 classrooms could deploy monitoring in every room for about INR 67,500 roughly the cost of two or three traditional commercial data loggers. The ESP32's Wi-Fi capability means no additional networking infrastructure beyond the institution's existing Wi-Fi is needed. The thing Speak free tier supports up to 8 fields at 15-second minimum intervals, which is sufficient for this application. For a larger deployment with many nodes, alternatives like Influx DB with Grafana (self-hosted) or the MathWorks academic license would be more appropriate. If deploying in locations without institutional Wi-Fi coverage, LoRa or NB-IoT communication modules could replace the ESP32's Wi-Fi layer though at additional cost.

VII. CONCLUSION AND FUTURE WORK

This paper set out to answer a simple question: what are the actual thermal and air quality conditions inside a typical classroom in Gurugram NCR, and how do they compare to what ASHRAE defines as comfortable? The

answer, based on six months of continuous IoT monitoring, is that conditions are uncomfortable for roughly 77% of the observation period, with peak discomfort during the monsoon months of July and August when temperature and humidity are simultaneously elevated. The system I built to collect this data is simple, cheap (approximately INR 1,350), and reproducible. The ESP32 microcontroller combined with DHT22, DS18B20, and MQ-135 sensors provides a useful multiparameter view of the indoor environment, and the Thing Speak cloud platform makes it straightforward to store, visualise, and export data. The prototype achieved 97.3% data transmission success and 98.6% uptime over the validation period. The contributions of this work are: (1) a detailed, reproducible hardware design for a low-cost indoor environmental monitoring node; (2) a six-month labelled dataset from a real Indian academic classroom covering the full range of NCR seasonal conditions; (3) quantitative characterisation of thermal discomfort patterns against ASHRAE 55 thresholds; and (4) an honest assessment of the system's limitations.

For future work I would like to pursue several directions. First, expanding to a multi-node deployment with five nodes per room to capture spatial variability and generate indoor comfort maps. Second, replacing the MQ-135 with a calibrated NDIR CO2 sensor such as the MH-Z19B. Third, integrating a PIR occupancy sensor to correlate comfort parameters with room usage patterns. Fourth, developing an automated alert system that sends notifications when temperature or AQI exceeds thresholds, enabling real-time intervention. Fifth, using the collected time-series data to train a simple predictive model that forecasts classroom conditions an hour ahead based on time of day, day of week, and outdoor weather which could support proactive scheduling and ventilation decisions. The code and dataset for this project will be made available on my GitHub repository.

ACKNOWLEDGMENT

I would like to sincerely thank my project guide, [Dr. Digvijay Singh], for the encouragement and direction throughout this project, particularly during the early phase when I kept running into issues with the MQ-135 calibration. Thanks, are also due to the institution's IT department for providing a dedicated Wi-Fi SSID for the monitoring device. I also want to acknowledge the open-source Arduino and ESP32 community whose libraries, forum posts, and worked examples saved me countless hours of debugging. This project was supported by the Department of B Tech Communication Engineering AI&ML, [K. R. Mangalam University], as part of the undergraduate pre final-year project programme (Batch 2023–2027).

REFERENCES

[1] ASHRAE Standard 55-2020, "Thermal Environmental Conditions for Human Occupancy," American Society of Heating, Refrigerating and Air-Conditioning Engineers, Atlanta, GA, 2020.

[2] ISO 7730:2005, "Ergonomics of the Thermal Environment — Analytical Determination and Interpretation of Thermal Comfort Using PMV and PPD Indices," ISO, Geneva, 2005.

[3] A. Maier, A. Sharp, and Y. Vagapov, "Comparative Analysis and Practical Implementation of the ESP32 Microcontroller Module for the Internet of Things," in Proc. ITA, 2017, pp. 143–148.

[4] Y. A. Ahmad, A. H. Salameh, and A. A. Karim, "On the Evaluation of DHT22 Temperature Sensor for IoT Application," in Proc. ICCCE 2021, pp. 131–134, doi: 10.1109/ICCCE50029.2021.9467147.

[5] L. M. Easterline et al., "Smart Air Quality Monitoring with IoT-based MQ-Series Sensors and ESP32 with ThingSpeak Integration," Procedia Computer Science, vol. 245, pp. 815–824, 2024.

[6] A. Albita and D. Selişteanu, "A Compact IIoT System for Remote Monitoring and Control," Sensors, vol. 23, no. 4, p. 1784, 2023, doi: 10.3390/s23041784.

[7] M. Babiuch and J. Postulka, Smart Home Monitoring System with ESP32 Microcontrollers. London: IntechOpen, 2021.

[8] S. Anshu, R. Rawal, and A. Singh, "Ranking of Indian Cities Based on Heat Index for Mixed Mode Buildings," Environmental Science and Pollution Research, 2023.

[9] Centre for Science and Environment, "Sharp Rise in Summer Heat and Humidity in Delhi Jacks Up Cooling Demand," CSE India, New Delhi, 2024. [Online]. Available: https://www.cseindia.org

[10] T. Matthews et al., "Indoor Heat Measurement Data from Low-Income Households in Rural and Urban South Asia," Scientific Data, vol. 9, no. 278, 2022, doi: 10.1038/s41597-022-01314-5.

[11] Delhi Disaster Management Authority, "Delhi Heat Action Plan 2024–25," Govt. of NCT of Delhi, New Delhi, 2024.

[12] F. Gul and H. Eroglu, "Low-Cost IoT Mesh Network for Real-Time Indoor Air Quality Monitoring," in Proc. ICCES 2024, pp. 502–507, doi: 10.1109/ICCES63552.2024.10860109.

[13] N. Barot et al., "Enhanced IoT-Based Environmental Monitoring with MQTT and Predictive Analytics," IAR Journal of Engineering and Technology, vol. 6, no. 3, Jun. 2025.

[14] Ministry of Environment, Forest and Climate Change, "India Cooling Action Plan (ICAP)," Govt. of India, New Delhi, 2019.

[15] A. Andreiadis et al., "IoT-Based Real-Time Temperature Monitoring in Critical Systems: A Review," IgMin Research, May 2025.